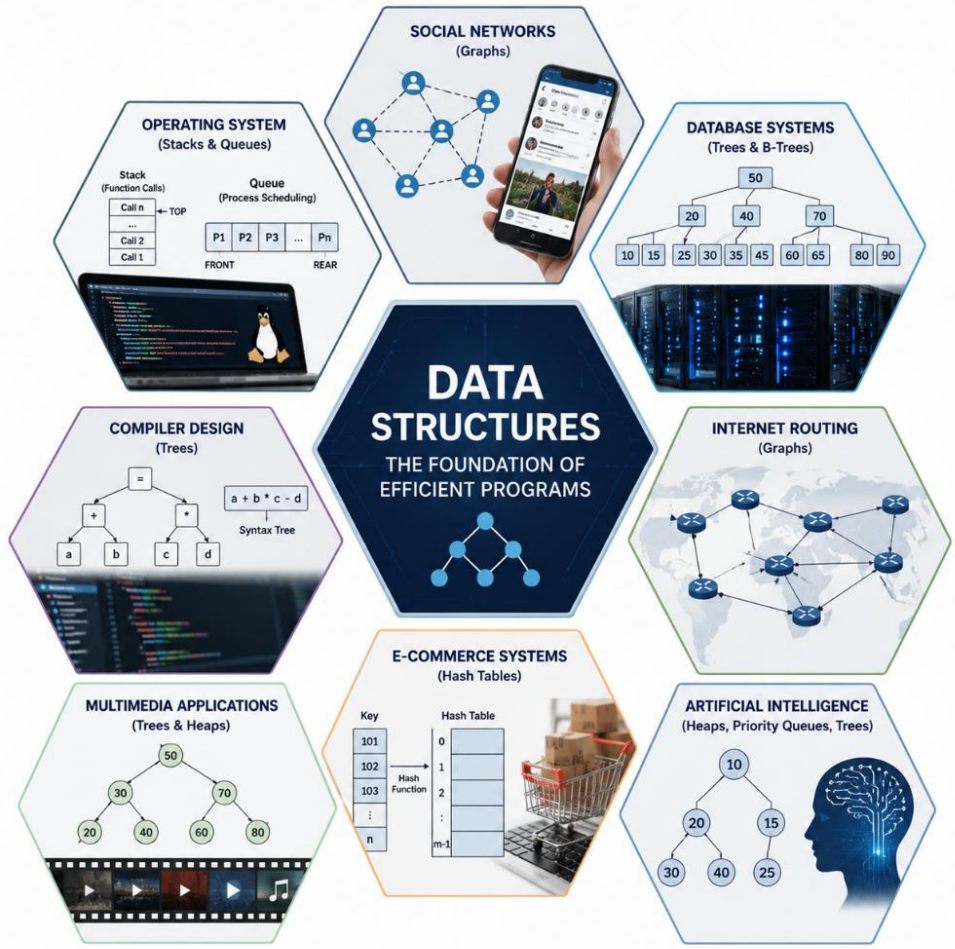
# Data Structure & Algorithm Lab

# ECE2132





Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

**Manipal University Jaipur,**

**Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)**

**Rajasthan 303007**


MANIPAL UNIVERSITY
JAIPUR
*(University under Section 2(f) of the UGC Act)*

INSPIRED BY LIFE

**(AY 2026-27)**

# Experiment No. 8

## Write a Program to Implement Quick Sort for Sorting an Array of Integers and Compare its Performance with Other Sorting Techniques

**AIM: To study and implement the Quick Sort algorithm in C++ for sorting an array of integers in ascending order and compare its performance with other sorting techniques.**

**SOFTWARE USED:** Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

**THEORY:** Sorting is the process of arranging data elements in a particular order. In this experiment, the elements of an integer array are arranged in ascending order using the Quick Sort algorithm.

Quick Sort is a comparison-based sorting algorithm. It follows the divide-and-conquer approach. The algorithm selects one element from the array as the pivot and divides the array into smaller parts. The elements smaller than the pivot are placed on the left side, while the elements greater than the pivot are placed on the right side. The pivot is then placed in its correct position in the sorted array. This process is repeated recursively for the left and right subarrays until all elements are sorted. Quick Sort is an efficient sorting algorithm and is suitable for sorting large arrays.

Its best-case and average-case time complexity is O(n log n), while the worst-case time complexity is $O(n^2)$. The space complexity is O(log n) on average due to recursive function calls.

Quick Sort

Quick Sort is a comparison-based sorting algorithm. It works using the divide-and-conquer technique.

The algorithm divides the array into two parts:

Left part – contains elements that are smaller than the pivot.

Right part – contains elements that are greater than the pivot.

The pivot is placed between these two parts at its correct position.

The same process is then applied recursively to the left and right parts until the entire array is sorted.

**Divide-and-Conquer Steps**

- Select a pivot element.
- Partition the array around the pivot.

- Place smaller elements before the pivot and larger elements after the pivot.
- Recursively apply Quick Sort to the left subarray.
- Recursively apply Quick Sort to the right subarray.
- Continue until the subarrays contain zero or one element.

**Example**

Consider the array:

A = [10, 7, 8, 9, 1, 5]

Initially:

[10, 7, 8, 9, 1, 5]

Select 5 as the pivot:

[10, 7, 8, 9, 1, 5]

↑

Pivot

Elements smaller than 5 are placed on the left and elements greater than 5 are placed on the right.

After partitioning:

[1, 5, 8, 9, 10, 7]

↑

Pivot

Since 5 is now in its correct position, the remaining elements are divided into smaller subarrays.

Left subarray: [1]

Right subarray: [8, 9, 10, 7]

Select 7 as the pivot:

[8, 9, 10, 7]

↑

Pivot

After partitioning:

[7, 9, 10, 8]

The same process is repeated for the remaining elements.

Similarly, the remaining elements are partitioned and sorted recursively.

Final sorted array:

[1, 5, 7, 8, 9, 10]

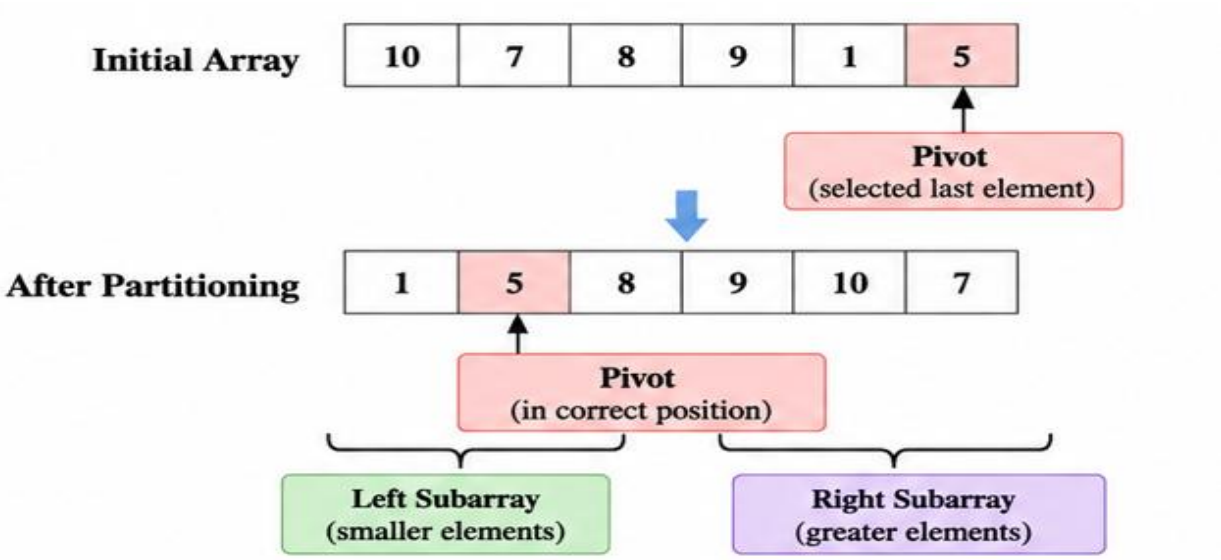



Figure. 1: Partitioning in Quick Sort

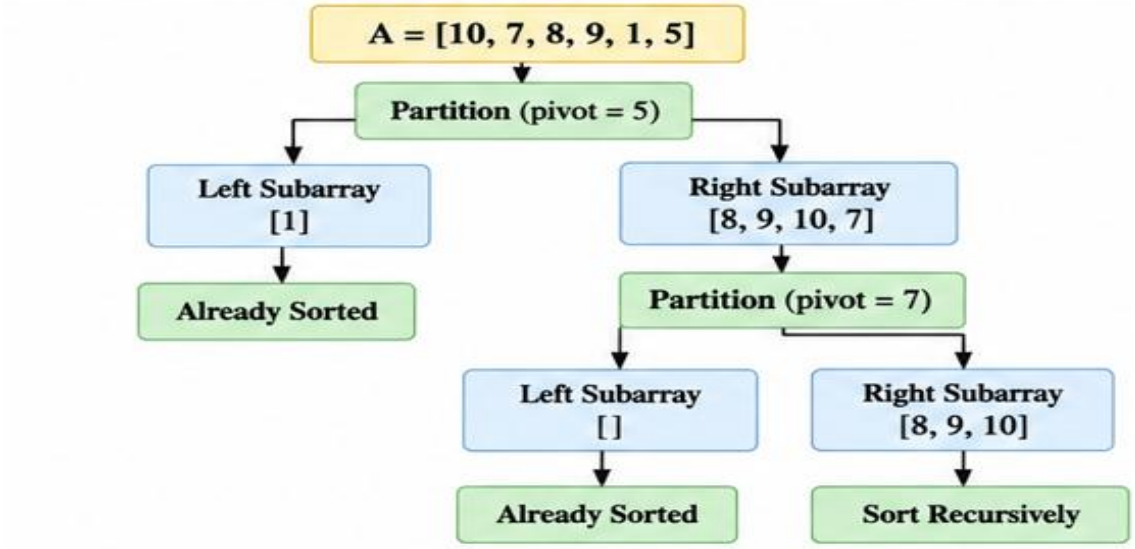



Figure. 2: Recursive Division in Quick Sort

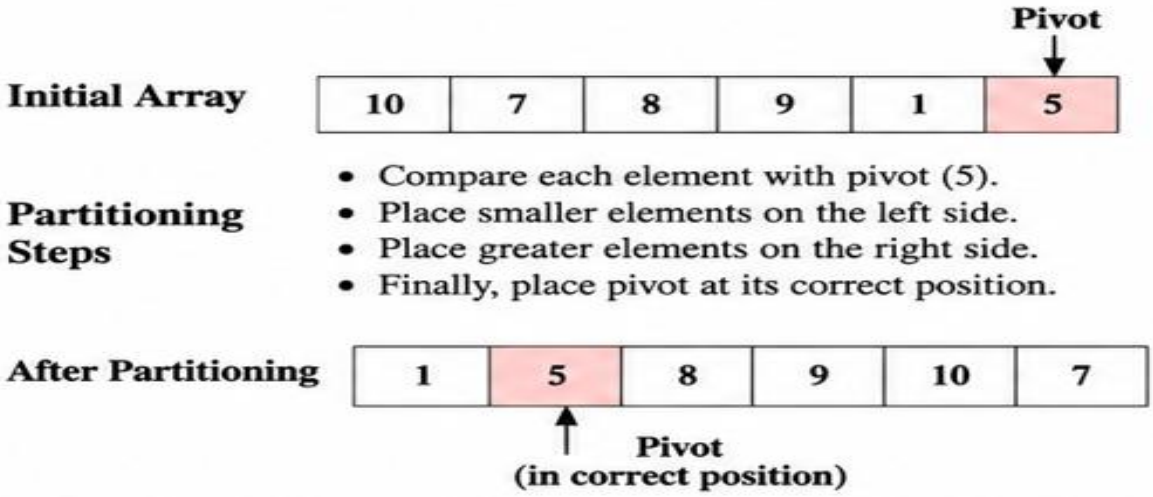



Figure. 3: Working of Quick Sort

| Feature | Quick Sort | Insertion Sort | Bubble Sort | Selection Sort |
|---|---|---|---|---|
| Type | Comparison Sort | Comparison Sort | Comparison Sort | Comparison Sort |
| Best Case Time Complexity | O(n log n) | O(n) | O(n) | $O(n^2)$ |
| Average Case Time Complexity | O(n log n) | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Worst Case Time Complexity | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Space Complexity | O(log n) | O(1) | O(1) | O(1) |
| Stability | Unstable | Stable | Stable | Unstable |
| In-Place | Yes | Yes | Yes | Yes |

Figure. 4: Comparison with Other Sorting Algorithms

**ALOGRITHM:**

1. Start.
2. Declare an integer array arr and enter the number of elements n.
3. Enter the elements of the array.
4. Select an element as the pivot.
5. Partition the array around the pivot.
6. Place the pivot in its correct position.
7. Recursively apply Quick Sort to the left subarray.
8. Recursively apply Quick Sort to the right subarray.
9. Repeat until the subarrays contain zero or one element.
10. Display the sorted array.
11. Apply other sorting algorithms to the same input.
12. Measure and compare their execution times.
13. Stop

## PROGRAM (C++):

```
#include <iostream>
using namespace std;

// Function to swap two elements
void swapElements(int &a, int &b)
{
     int temp = a;
     a = b;
     b = temp;
}

// Function to partition the array
int partitionArray(int arr[], int low, int high)
{
     int pivot = arr[high];      // Last element as pivot
     int i = low - 1;
     for (int j = low; j < high; j++)
     {
          if (arr[j] < pivot)
          {
               i++;
               swapElements(arr[i], arr[j]);
          }
     }
     swapElements(arr[i + 1], arr[high]);
     return i + 1;              // Return pivot index
}

// Function to implement Quick Sort
void quickSort(int arr[], int low, int high)
{
     if (low < high)
     {
          int pi = partitionArray(arr, low, high);
          quickSort(arr, low, pi - 1);    // Sort left subarray
          quickSort(arr, pi + 1, high);  // Sort right subarray
     }
}

// Function to display the array
void displayArray(int arr[], int n)
{
     for (int i = 0; i < n; i++)
     {
          cout << arr[i] << " ";
     }
     cout << endl;
}

int main()
{
     int n;
     cout << "Enter number of elements: ";
     cin >> n;

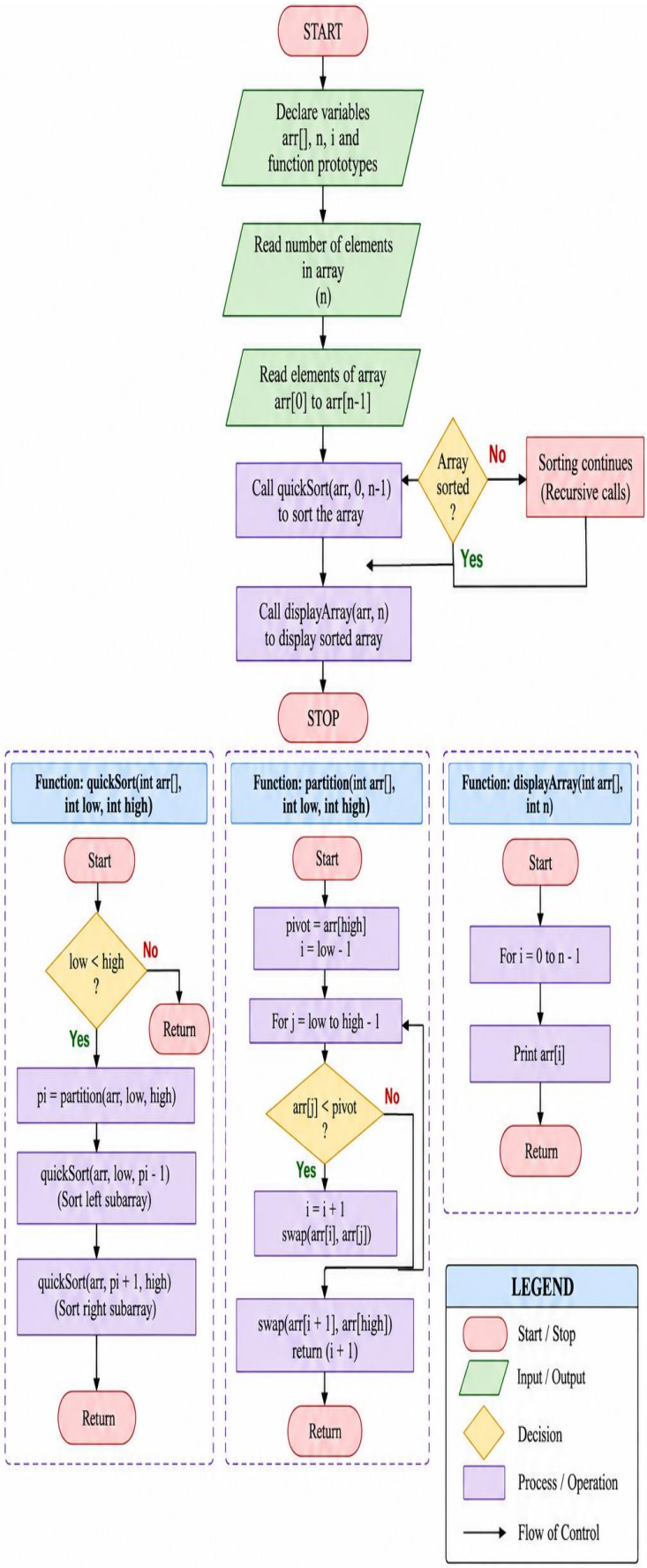
     int arr[n];
     cout << "Enter elements of the array:" << endl;
     for (int i = 0; i < n; i++)
     {
          cin >> arr[i];
     }

     cout << "Original Array: ";
     displayArray(arr, n);

     quickSort(arr, 0, n - 1);

     cout << "Sorted Array: ";
     displayArray(arr, n);

     return 0;
}
```



**EXPLANATION OF THE PROGRAM:**

| Statement / Function | Explanation |
|---|---|
| swapElements() | Swaps two elements of the array. |
| partitionArray() | Selects the pivot and partitions the array. |
| quickSort() | Recursively sorts the left and right subarrays. |
| bubbleSort() | Sorts the array using Bubble Sort. |
| selectionSort() | Sorts the array using Selection Sort. |
| insertionSort() | Sorts the array using Insertion Sort. |
| copyArray() | Creates separate copies of the original array. |
| displayArray() | Displays the array elements. |
| clock() | Measures the execution time of a sorting algorithm. |
| pivot | Element used to divide the array during partitioning. |
| low / high | Starting and ending indexes of the current subarray. |
| pi | Final position of the pivot after partitioning. |

**SAMPLE INPUT 1:**

Enter number of elements:

6

Enter elements of the array:

10 7 8 9 1 5

**CALCULATION:**

Initial array:

10 7 8 9 1 5

Pass 1

Pivot = 5

10 7 8 9 1 5

Elements smaller than 5 are placed on the left side.

1 is smaller than 5, so it is moved to the left.

After partitioning:

1 5 8 9 10 7

Pivot 5 is now in its correct position.

**Left subarray:**

1

**Right subarray:**

8 9 10 7

Pass 2

Pivot = 7

**Right subarray:**

8 9 10 7

No element is smaller than 7.

Pivot 7 is placed at the beginning of the subarray.

After partitioning:

7 9 10 8

Pivot 7 is now in its correct position.

Left subarray:

Empty

**Right subarray:**

9 10 8

Pass 3

Pivot = 8

9 10 8

No element is smaller than 8.

Pivot 8 is placed at its correct position.

**After partitioning:**

8 10 9

Pivot 8 is now in its correct position.

Right subarray:

10 9

Pass 4

Pivot = 9

10 9

No element is smaller than 9.

Pivot 9 is placed at its correct position.

**After partitioning:**

9 10

Pivot 9 is now in its correct position.

**Final Sorted Array:**

1 5 7 8 9 10

Therefore, the final sorted array is:

1 5 7 8 9 10

**Sample Output:**

Enter number of elements: 6

**Enter elements of the array:**

10 7 8 9 1 5

**Sorted Array using Quick Sort:**

1 5 7 8 9 10

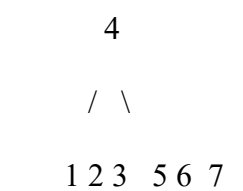
**PERFORMANCE ANALYSIS:**

Quick Sort performance depends on the selection of the pivot and the initial arrangement of the elements in the array.

Best Case

The best case occurs when the pivot divides the array into two nearly equal parts.

Example:




In this case, the array is divided into nearly equal subarrays at each step. The number of comparisons is reduced and the algorithm performs efficiently.

Time Complexity = O(n log n)

**Average Case**

In the average case, the elements are in a random order. The pivot generally divides the array into reasonably balanced subarrays.

The algorithm recursively sorts the left and right subarrays.

Time Complexity = O(n log n)

**Worst Case**

The worst case occurs when the pivot is always the smallest or largest element.

For example, when the array is already sorted:

1 2 3 4 5 6

Since the program selects the last element as the pivot, each partition may produce one empty subarray and one subarray containing the remaining elements.

The same process is repeated for the remaining elements.

Time Complexity = $O(n^2)$

**TIME COMPLEXITY:**

Best Case: O(n log n)

Average Case: O(n log n)

Worst Case: $O(n^2)$

**For an array containing n elements:**

Best case: The array is divided into nearly equal subarrays.

Average case: The array is divided into reasonably balanced subarrays.

Worst case: The pivot produces highly unbalanced partitions, resulting in approximately $n^2$ comparisons.

Therefore, the overall worst-case time complexity of Quick Sort is:

$O(n^2)$

Space Complexity

Quick Sort is an in-place sorting algorithm because it sorts the elements within the original array without requiring a separate array for storing the sorted elements.

However, the algorithm uses recursive function calls.

For balanced partitions, the recursion depth is approximately log n.

Average Space Complexity = O(log n)

In the worst case, the recursion depth can become n.

Worst Space Complexity = O(n)

**PERFORMANCE COMPARISON:**

| Sorting Algorithm | Best Case | Average Case | Worst Case |
|---|---|---|---|
| Quick Sort | O(n log n) | O(n log n) | $O(n^2)$ |
| Bubble Sort | O(n) | $O(n^2)$ | $O(n^2)$ |
| Selection Sort | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Insertion Sort | O(n) | $O(n^2)$ | $O(n^2)$ |

**CHARACTERISTICS OF INSERTION SORT:**

1. It is a comparison-based sorting algorithm.
2. It follows the divide-and-conquer approach.
3. It uses a pivot element to partition the array.
4. Elements smaller than the pivot are placed on the left side.
5. Elements greater than the pivot are placed on the right side.
6. It is an in-place sorting algorithm.
7. Its average-case time complexity is O(n log n).
8. Its worst-case time complexity is $O(n^2)$.
9. Its average space complexity is O(log n) due to recursion.
10. Its performance can be improved by choosing a suitable pivot.

## IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;

// Function to swap two elements
void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Function to partition the array
int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

// Function to implement Quick Sort
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for(int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

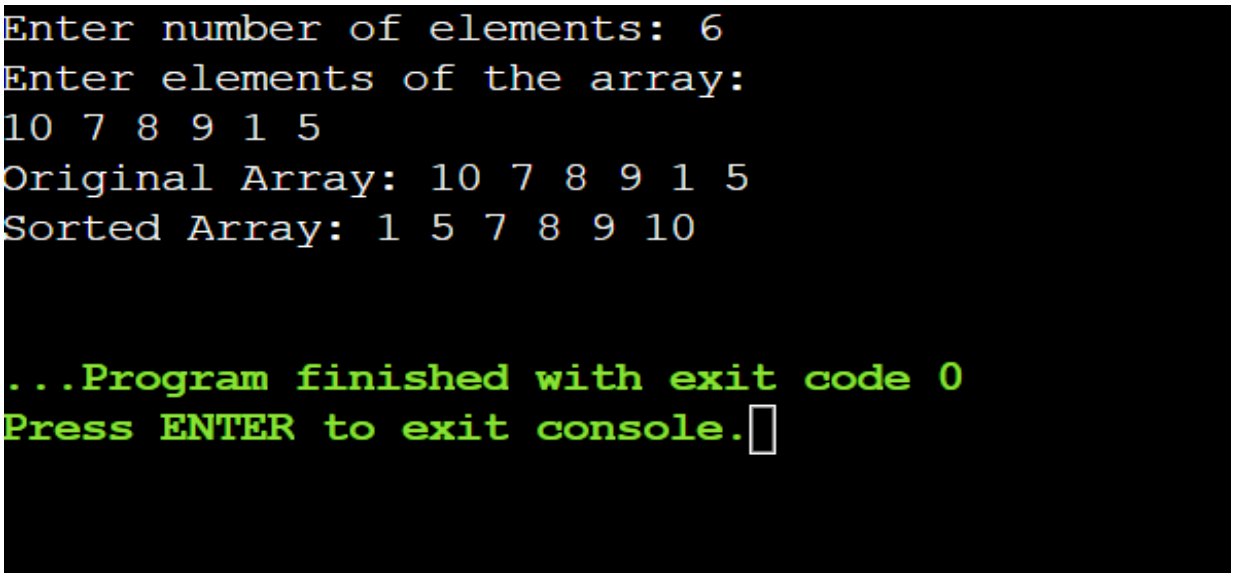
    int arr[n];
    cout << "Enter elements of the array:" << endl;
    for(int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    cout << "Original Array: ";
    displayArray(arr, n);

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```




**APPLICATIONS**: Quick Sort can be used in sorting large datasets, database systems, searching and data processing applications, computer systems, numerical and scientific applications, file and record management, and general-purpose sorting.

**RESULT:** The program was successfully executed. It demonstrated the implementation of Quick Sort in C++ for sorting an array of integers in ascending order. The performance of Quick Sort was compared with Bubble Sort, Selection Sort, and Insertion Sort, and their time complexities were studied.

**ASSIGNMENT:**

- WAP to implement Quick Sort for sorting an array in descending order.
- WAP to implement Quick Sort using separate functions for input, partitioning, sorting, and display.
- WAP to implement Quick Sort using the first element as pivot.
- WAP to implement Quick Sort using the middle element as pivot.
- WAP to count the number of comparisons performed by Quick Sort.
- WAP to count the number of swaps performed by Quick Sort.
- Compare Quick Sort performance for sorted, reverse-sorted, and randomly arranged arrays.
- Compare Quick Sort with Merge Sort for different array sizes.